Architectural Memory and Computational Bottlenecks in CNN Training in Julia

Michał Bibrzycki
Faculty of Electrical Engineering
Warsaw University of Technology
Warsaw, Poland

Mikołaj Tradecki
Faculty of Electrical Engineering
Warsaw University of Technology
Warsaw, Poland

***Abstract*—Modern deep learning frameworks often prioritize flexibility over raw performance, leading to significant memory and computational overhead on CPUs. This paper investigates these bottlenecks, focusing on dynamic memory allocation and type instability in standard Automatic Differentiation (AD) engines. To address this, we developed a custom AD framework in Julia from scratch. By using a single, pre-allocated memory pool and flat array indexing, our architecture bypasses the Garbage Collector (GC) during the training loop. We validated our framework by training a Convolutional Neural Network (CNN) on the FashionMNIST dataset and benchmarked it against Flux.jl and PyTorch. Our results show that this zero-allocation approach drastically reduces memory overhead and execution time, demonstrating that strict, static memory management is highly effective for CPU-bound deep learning.**

***Index Terms*—Automatic Differentiation, Deep Learning Optimization, Zero-Allocation Architecture, Julia Language.**

I. Introduction

Convolutional Neural Networks (CNNs) have established themselves as the primary architecture for image recognition and computer vision tasks [1]. While the training of these complex models is typically accelerated using Graphics Processing Units (GPUs), CPU-bound optimization remains a critical field of study. Efficient CPU execution is essential for edge computing, embedded devices, and environments where dedicated hardware accelerators are either unavailable or cost-prohibitive [2].

Modern deep learning frameworks are designed to prioritize flexibility and ease of use. They achieve this by employing dynamic computation graphs and generalized Automatic Differentiation (AD) engines [3], [4]. However, this flexibility introduces significant computational bottlenecks when executed on standard processors. The continuous allocation of temporary arrays, dynamic type dispatch, and the resulting Garbage Collection (GC) overhead create a memory wall that drastically degrades training performance.

In this paper, we address these inefficiencies by proposing a custom, zero-allocation AD framework written in Julia. We demonstrate that by replacing dynamic tensor abstractions with a static computational graph and a centralized, pre-allocated memory pool, we can entirely bypass GC interventions during the optimization loop. The framework's mathematical correctness and performance are evaluated by training a CNN on the FashionMNIST dataset [5].

II. Literature

A. Automatic Differentiation and Memory Overheads

The foundation of training Convolutional Neural Networks (CNNs) lies in the backpropagation algorithm [6], which computes the gradient of a loss function with respect to the network's weights. Modern frameworks automate this process using Reverse-Mode Automatic Differentiation (AD) [7]. While AD greatly simplifies model development, it inherently requires storing intermediate activation values from the forward pass to compute gradients during the backward pass. This storage requirement often leads to significant memory consumption and allocation overheads [8].

On CPU architectures, performance is heavily bottlenecked by memory bandwidth and cache utilization rather than raw computational power [9]. Standard convolution operations, often implemented via the *im2col* transformation to leverage highly optimized matrix multiplication [10], typically require allocating temporary memory buffers for every batch. Without strict memory management, these continuous allocations trigger frequent Garbage Collection (GC) pauses, drastically reducing overall training throughput.

B. Dynamic vs. Static Computation Graphs

Historically, deep learning libraries utilized static computation graphs compiled before execution, allowing for aggressive memory pre-allocation [11]. In contrast, modern tools like PyTorch [4] popularized the "define-by-run" dynamic graph paradigm. While highly flexible for developers, constructing computation graphs on-the-fly necessitates continuous heap allocations, measurably degrading CPU performance.

C. The Julia Ecosystem and AD Methodologies

The Julia programming language was designed to combine the high-level syntax of Python with the execution speed of compiled languages like C [12]. In the context of machine learning, Flux.jl [3] is the standard framework, relying on Zygote.jl [13] as its AD backend. Zygote operates as a source-to-source compiler, analyzing Julia code to generate gradient computations automatically.

This generalized approach struggles with in-place memory mutations and scalar iterations, forcing the system to allocate new arrays for intermediate operations. This leads to substantial GC overhead when executed on CPU environments. Our

work addresses this specific gap by returning to a compiled, static-graph approach combined with a centralized memory pool, bypassing the dynamic dispatch and allocation issues present in generalized AD solutions.

III. System Architecture and Layer Optimizations

This section analyzes general computational and memory bottlenecks encountered during the implementation of deep learning primitives on CPU architectures, highlighting the contrasts between naive patterns, industry-standard frameworks, and the proposed hardware-friendly implementation.

A. Memory Layouts and Contiguous Indexing

Naive implementations of computational graphs allocate memory for weights, gradients, and activations locally within each node. This approach creates unnecessary allocations and limits cache locality. PyTorch mitigates this by storing weights persistently within each node and utilizing a caching allocator for activations and gradients to recycle freed buffers. This is highly effective for large-scale networks where static pre-allocation is infeasible and allocation costs are amortized over large kernels. Conversely, Flux relies heavily on Julia's native garbage collection and LLVM optimizations without a dedicated system for cache locality.

To eliminate GC overhead entirely for smaller topologies, a centralized memory architecture can be employed. By allocating all weights, gradients, and activations into large, contiguous one-dimensional vectors during an initial build step, cache locality is maximized. This ensures strictly zero allocations during the hot training loop, though it is primarily feasible for networks where all parameters fit within available contiguous memory.

B. Type Instability and Dynamic Dispatch

A standard approach to dynamic computational graphs involves storing network nodes in arrays as boxed pointers, which introduces significant runtime overhead due to dynamic type resolution. PyTorch incurs this dynamic dispatch penalty at both the Python level and within its C++ backend. However, this overhead becomes negligible for computationally intensive operations on large inputs. Flux bypasses dynamic dispatch by structuring the network as a statically typed `Tuple`, leveraging Julia's multiple dispatch system.

To further optimize this in a static framework, Julia's metaprogramming capabilities (e.g. `@generated` functions) can be utilized to completely unroll the forward and backward execution loops. This translates the network graph into explicit, statically-dispatched sequential function calls (such as `primal!` and `adjoint!`), ensuring stable type inference and eliminating runtime dispatch overhead.

C. Convolutional Layer

The convolution operation consumes most of the computational time. Utilizing the *im2col* transformation allows the forward pass to be computed via a single GEMM (General Matrix Multiply) call provided by BLAS (Basic Linear Algebra Subprograms) — a standardized linear algebra interface with highly optimized implementations such as OpenBLAS, used by both Julia and PyTorch [10]. Because Julia is a column-major language [12], the spatial *im2col* loops should be structurally inverted to ensure *stride-1* memory access. We tried to explicitly eliminate optional operations (e.g. bias addition) by encoding it as a parametric type to allow the LLVM compiler to perform dead code elimination. This turned out to be counter-productive.

D. Max Pooling Layer

Pooling is memory-bound: each output requires only a few comparisons, so memory access patterns dominate the cost. During the forward pass, each window's argmax is recorded as a single packed `Int32` instead of a heap-allocated index mask. This bookkeeping makes the forward pass slower than Flux's, but reduces the backward pass to a sparse scatter into pre-recorded positions, so the combined forward–backward cost outperforms both Flux and PyTorch.

E. Flatten Layer

Transitioning from convolutional feature maps ($H \times W \times C$) to fully connected layers requires flattening. If the underlying memory architecture intrinsically stores all data as flat 1D arrays, the flatten operation bypasses expensive data permutations. Thanks to identical column-major memory layout, flatten resolves to a reshape operation with just changing index information on the view wrapper without actually moving any data.

F. Dense Layer

The fully connected (Dense) layer computes $\mathbf{Y} = \mathbf{W}\mathbf{X} + \mathbf{b}$. Optimization in this layer strictly targets the minimization of memory permutations. By ensuring the weight matrix $\mathbf{W}$ is allocated in a layout that aligns with the OpenBLAS column-major execution path, hidden, on-the-fly matrix transpositions are avoided, allowing for peak single-threaded CPU throughput.

G. ReLU Layer

The Rectified Linear Unit, defined as $f(x) = \max(0, x)$ [1], requires knowing which neurons were active during backpropagation. Standard implementations store a boolean mask from the forward pass, costing an extra buffer and an extra write per element. Instead, the gradient condition is re-evaluated on-the-fly from the already-resident activations using branchless `ifelse` instructions, eliminating both the mask storage and the associated memory traffic.

H. Dropout Layer

Dropout mitigates overfitting by randomly zeroing activations [1]. The implementation uses inverted dropout: the $1/(1-p)$ scaling is applied at training time, reducing the inference path to a plain copy. No boolean mask is stored — the backward pass re-evaluates the keep condition from the same random buffer, which remains untouched between the forward and backward passes, saving one buffer and one write per element.

I. Softmax Layer

Evaluating Softmax and Cross-Entropy sequentially generates unnecessary temporary arrays. Fusing them into a single layer eliminates this inefficiency. After subtracting the maximum logit z_{max} for numerical stability, the gradient simplifies to $\frac{\partial L}{\partial z_i} = \sigma(\mathbf{z})_i - y_i$. Although intermediate probabilities are stored in a static memory pool, this allocation-free execution completely avoids dynamic heap allocations, tightly bounding the memory footprint and reducing computational overhead.

IV. Experimental Setup

To ensure reproducible benchmarking, all experiments were conducted in the same CPU environment. The evaluations were performed on an AMD Ryzen 7 processor. The CPU features 8 physical cores and 16 hardware threads, operating with a maximum boost frequency of 5.1 GHz. To measure the raw, single-threaded mathematical throughput and expose the architectural overheads of the frameworks, the BLAS threading was restricted to a single thread.

The models were trained on the FashionMNIST dataset [5], utilizing a batch size of 10. The benchmarking isolates the performance of our custom zero-allocation Automatic Differentiation (AD) engine against two industry-standard dynamic frameworks: Flux.jl and PyTorch.

V. Results and Evaluation

This section presents a layer-by-layer micro-benchmarking analysis. Both execution time and memory allocations are compared, with operations divided into the forward and backward passes. In dynamic frameworks (Flux.jl, PyTorch), layers dynamically allocate intermediate tensors during the forward pass to build the computational graph. In contrast, the proposed custom engine routes all data through a static, pre-allocated memory pool.

A. Memory Pre-allocation and Initialization

Dynamic frameworks generate almost zero cost prior to the training loop, as tensors are allocated on the fly. However, our custom architecture forces a one-time, upfront allocation phase where the `MemoryPool` reserves contiguous 1D arrays for all weights, gradients, and activations. Table I demonstrates this initial overhead, which pays off during the execution loop.

Table I
Memory Pre-allocation Benchmark

Framework	Time (ms)	Allocated Memory (KiB)
Ours	2.138	7 656.4
Flux.jl	0.081	533.0
PyTorch	1.695	264.5

B. Convolutional Layer

The convolution operation highlights the difference in memory layout strategies. For the first convolutional layer (3×3, $1 \rightarrow 6$ channels), our approach executes the forward pass in 30.72 μs with zero allocations. In contrast, Flux and PyTorch incur significant heap allocations (> 180 KiB) and longer execution times due to intermediate tensor instantiations during their convolution lowering.

Table II
Convolutional Layer Benchmark

Framework	Pass	Time (μs)	Alloc. (KiB)
Ours	Forward	30.72	0
	Backward	42.25	0
Flux.jl	Forward	196.98	217.88
	Backward	329.35	88.57
PyTorch	Forward	101.00	183.75
	Backward	374.71	30.84

C. Max Pooling Layer

While the custom Max Pooling layer eliminates allocations entirely, its forward pass relies on JIT-compiled scalar loop unrolling, which is outperformed by Flux. However, this initial time cost is amortized during the backward pass (11.90 μs vs 212.03 μs), where our framework avoids allocating integer index masks on the heap.

Table III
Max Pooling Layer Benchmark

Framework	Pass	Time (μs)	Alloc. (KiB)
Ours	Forward	126.17	0
	Backward	11.90	0
Flux.jl	Forward	17.44	51.71
	Backward	212.03	185.20
PyTorch	Forward	204.96	137.81
	Backward	38.27	91.88

D. Flatten Operation

Flattening a tensor in generic AD engines often forces a memory copy or creates a complex view object that must be tracked by the tape. By relying on a globally flat 1D memory layout, our flatten operation reduces to a zero-allocation operation taking 0.02 μs.

Table IV
Flatten Layer Benchmark

Framework	Pass	Time (μs)	Alloc. (KiB)
Ours	Forward	0.02	0
	Backward	0.02	0
Flux.jl	Forward	4.31	1.05
	Backward	0.72	0.20
PyTorch	Forward	5.27	0
	Backward	18.11	0

E. Dense Layer

The Dense layer relies heavily on GEMM routines. PyTorch maintains an execution speed advantage during the backward pass (76.13 μs vs 113.58 μs). Nevertheless, our engine remains highly competitive, circumventing the $\sim$289 KiB gradient allocation overhead present in both dynamic counterparts.

Table V
DENSE LAYER BENCHMARK

Framework	Pass	Time (μs)	Alloc. (KiB)
Ours	Forward	12.05	0
	Backward	113.58	0
Flux.jl	Forward	19.60	6.90
	Backward	355.53	289.03
PyTorch	Forward	43.72	3.28
	Backward	76.13	288.20

F. ReLU Activation

Standard implementations cache a boolean mask tensor during the forward pass to compute gradients later. Our engine avoids the mask tensor entirely by re-evaluating the max-condition directly from the pre-allocated activation arrays during backpropagation, keeping execution times under 0.10 μs.

Table VI
RELU LAYER BENCHMARK

Framework	Pass	Time (μs)	Alloc. (KiB)
Ours	Forward	0.09	0
	Backward	0.10	0
Flux.jl	Forward	8.68	4.29
	Backward	1.51	3.54
PyTorch	Forward	10.28	3.28
	Backward	16.36	3.28

G. Dropout Layer

Dropout tests the efficiency of random number generation. While dynamic engines allocate fresh tensors for the random noise, our engine overwrites a single pre-allocated buffer in place, operating roughly 20x faster than Flux in the forward pass.

Table VII
DROPOUT LAYER BENCHMARK

Framework	Pass	Time (μs)	Alloc. (KiB)
Ours	Forward	0.38	0
	Backward	0.11	0
Flux.jl	Forward	8.97	9.30
	Backward	4.75	3.57
PyTorch	Forward	33.90	6.56
	Backward	20.10	0

H. Softmax Layer

Evaluating Softmax and Cross-Entropy sequentially generates unnecessary intermediate probability tensors. The proposed custom layer fuses these operations, performing numerical stability corrections and gradient computations in a single, allocation-free loop that executes in 0.07 μs during backpropagation.

Table VIII
FUSED SOFTMAX/LOSS BENCHMARK

Framework	Pass	Time (μs)	Alloc. (KiB)
Ours	Forward	1.03	0
	Backward	0.07	0
Flux.jl	Forward	6.70	3.78
	Backward	3.09	2.30
PyTorch	Forward	18.54	0.40
	Backward	51.12	$\sim$0

I. Overall Training Pass Performance

The cumulative effect of eliminating dynamic dispatch, branch conditions, and heap allocations becomes apparent when measuring a complete forward and backward pass for the entire network (3 epochs). Table IX summarizes the macro-level benchmark. It illustrates that our static-graph architecture operates faster than both PyTorch and Flux, while achieving zero-allocation runtime.

Table IX
FULL TRAINING BENCHMARK

Framework	Time (s)	Total Allocations (GiB)
Ours	13.12	0
Flux.jl	38.77	24.236
PyTorch	18.30	$\sim$18.6

VI. CONCLUSION

This paper presented a custom, zero-allocation Automatic Differentiation framework in Julia, designed for efficient CPU training. By using a static computational graph and a single pre-allocated memory pool, the Garbage Collection (GC) overhead was eliminated during the main training loop.

Benchmarks on the FashionMNIST dataset showed that this approach drastically reduces memory allocations and execution time compared to Flux.jl and PyTorch. These results confirm the initial hypothesis: strict memory management and hardware-aware optimizations (such as stride-1 memory access and branchless execution) are highly effective for accelerating deep learning on standard processors.

REFERENCES

[1] I. Goodfellow, Y. Bengio, and A. Courville, "Deep Learning," *MIT Press*, 2016.

[2] J. Chen and X. Ran, "Deep Learning With Edge Computing: A Review," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1655–1674, 2019.

[3] M. Innes, E. Saba, K. Fischer, D. Gandhi, M. C. Rudilosso, N. M. Joy, T. Karmali, A. Pal, and V. Shah, "Fashionable Modelling with Flux," *arXiv preprint arXiv:1811.01457*, 2018.

[4] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[5] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms," *arXiv preprint arXiv:1708.07747*, 2017.

[6] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.

[7] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: a survey," *Journal of Machine Learning Research*, vol. 18, no. 153, pp. 1–43, 2018.

[8] T. Chen, B. Xu, C. Zhang, and C. Guestrin, "Training deep nets with sublinear memory cost," *arXiv preprint arXiv:1604.06174*, 2016.

[9] E. Georganas, S. Avancha, K. Banerjee, D. Kalamkar, G. Henry, H. Pabst, and A. Heinecke, "Anatomy of High-Performance Deep Learning Convolutions on SIMD Architectures," in *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*, IEEE, pp. 830–841, 2018.

[10] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, "cuDNN: Efficient Primitives for Deep Learning," *arXiv preprint arXiv:1410.0759*, 2014.

[11] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: A System for Large-Scale Machine Learning," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pp. 265–283, 2016.

[12] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, "Julia: A Fresh Approach to Numerical Computing," *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017.

[13] M. Innes, "Don't Unroll Adjoint: Differentiating SSA-Form Programs," *arXiv preprint arXiv:1810.07951*, 2018.